🏗️ Desafíos de Diseño de Sistemas

Esta sección contiene **desafíos prácticos** de nivel intermedio-avanzado centrados en el diseño de arquitecturas escalables, mantenibles y de alto rendimiento. Son ideales para practicar para entrevistas en empresas como Google, Amazon, Meta o startups con alta carga técnica.

- 🔹 Cada desafío plantea un problema abierto, con posibles múltiples soluciones.
- 🔹 Se incluye una guía de resolución paso a paso y consideraciones clave.
- 🔹 El foco no es programar, sino **pensar como un arquitecto de software**.

📌 Índice de Desafíos

1. 📦 Diseñar un sistema tipo Dropbox (Almacenamiento de Archivos Distribuido)

---

1. 📦 Diseñar un sistema tipo Dropbox (Almacenamiento de Archivos Distribuido)

🎯 Enunciado

Diseña **un sistema distribuido para almacenamiento y sincronización de archivos** similar a **Dropbox**, que permita a los usuarios:

- Subir, descargar, eliminar y compartir archivos.
- Sincronizar archivos entre múltiples dispositivos.
- Tener acceso offline y actualizaciones en segundo plano.
- Manejar versiones de archivos.

Requisitos no funcionales

- Escalabilidad global.
- Alta disponibilidad.
- Tolerancia a fallos.
- Latencia baja en lectura/escritura.
- Seguridad de datos y acceso.

🧠 Pasos de Resolución

1. **Recolección de Requisitos**

- Usuarios pueden crear cuentas y carpetas personales.

```
- Acciones básicas: CRUD de archivos.
- Capacidad de compartir archivos (privado/público).
- Sincronización casi en tiempo real.
- Control de versiones de archivos.
- Sistema debe escalar para millones de usuarios.

#### 2. **Estimaciones de Carga (opcional)**

- 100M usuarios activos.
- 1PB de datos almacenados.
- 10K archivos por segundo subidos.
- 50K lecturas por segundo.

💡 *Estas estimaciones guían decisiones sobre particionamiento, caché, base de
datos, etc.*

---

#### 3. **Diseño de Alto Nivel (HLD)**

##### Componentes principales

- **Client SDK** (para escritorio y móvil)
- **API Gateway**
- **Service Layer**
  - Auth Service
  - File Metadata Service
  - Upload/Download Service
  - Sync Service
  - Versioning Service
- **Storage Layer**
  - Metadata DB (SQL o NoSQL)
  - Blob Storage (ej. AWS S3, GCS)
- **Notification Queue** (para sincronización)
- **CDN** para archivos públicos

📌 **Diagrama sugerido** (dibujar fuera):

---

#### 4. **Diseño de Base de Datos**

##### Metadata DB

- Tabla de usuarios
- Tabla de archivos:
```

```
  - `file_id`, `user_id`, `path`, `version`, `timestamp`
- Tabla de permisos:
  - `file_id`, `shared_with_user`, `access_type`

##### Blob Storage

- Particionado por usuario o hash
- Replicación geo-distribuida

---

#### 5. **Manejo de Versiones**

- Cada modificación crea una nueva versión.
- Se puede usar almacenamiento incremental (delta encoding).
- Permitir al usuario restaurar versiones anteriores.

---

#### 6. **Sincronización entre dispositivos**

- Utiliza **WebSocket** o long-polling para notificar cambios.
- Cliente mantiene un log local de cambios.
- Back-end detecta conflictos y aplica resolución (last write wins / manual merge).

---

#### 7. **Optimización y Escalabilidad**

- **CDN** para acceso rápido a archivos públicos.
- **Chunked uploads** para archivos grandes.
- **Cache** de metadatos en Redis.
- **Rate limiting** para prevenir abuso.
- **Sharding** del Metadata Service por usuario ID.

---

#### 8. **Seguridad**

- Autenticación con OAuth 2.0.
- Criptografía en tránsito (TLS) y en reposo (AES).
- Validación de permisos en cada request.

---

### ✅ Consideraciones Finales
```

- ¿Qué pasa si dos dispositivos editan el mismo archivo offline?
- ¿Cómo diseñarías la detección de cambios en carpetas grandes?
- ¿Qué sistema de colas elegirías para los eventos de sincronización?
- ¿Cuáles servicios podrían desacoplarse en microservicios?

💡 Consejos para entrevistas

- Piensa en casos extremos y cuellos de botella.
- Justifica cada decisión: base de datos, colas, escalado, etc.
- Dibuja diagramas siempre que puedas.
- Evalúa trade-offs: consistencia vs disponibilidad, latencia vs complejidad.

📚 Recursos Recomendados

- [System Design Primer (GitHub)](https://github.com/donnemartin/system-design-primer)
- [Dropbox System Design - Gaurav Sen (YouTube)](https://www.youtube.com/watch?v=pebJvKkZJXg)
- [Designing Data-Intensive Applications (Martin Kleppmann)](https://dataintensive.net/)
- [Cloud Storage Architecture by Google](https://cloud.google.com/storage/docs/introduction)

Hecho con 📐 por [@holasoymalva](https://holasoymalva.com)